Backend Product Requirements Document (PRD)

Project: Obsidian

Version: v1.0

Architecture Scope: Production-Grade Backend System

Target Market: Indian Tech Startups & Remote Teams

Reference Source:

---

1. Executive Overview

Product Vision

Obsidian is a scalable virtual-office collaboration platform designed specifically for Indian startups requiring:

- Low operational cost
- Real-time communication
- Role-based organizational structures
- Persistent voice rooms
- Affordable team collaboration
- Discord-like UX for professional environments

The backend is engineered around a **serverless-first hybrid cloud architecture** using:

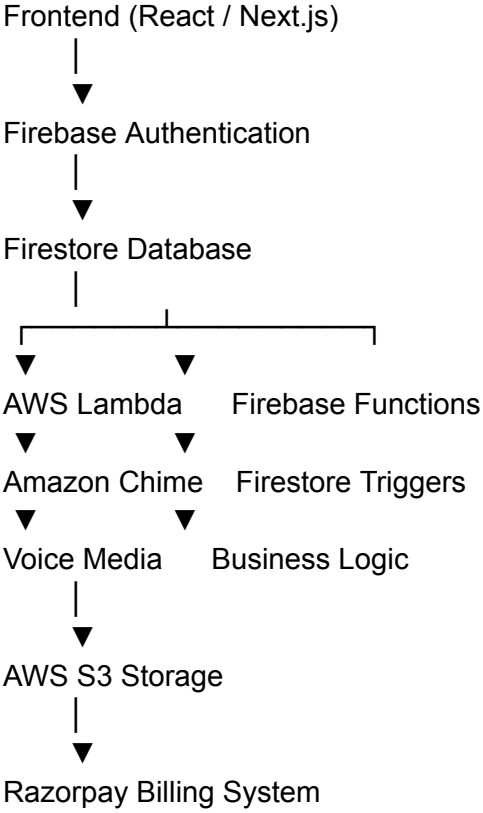
- Firebase Ecosystem
- AWS Media Infrastructure
- Razorpay Billing
- Event-Driven APIs

The system prioritizes:

- Cost optimization
- Real-time synchronization
- Minimal DevOps overhead
- Elastic scalability
- Security isolation
- Subscription-controlled infrastructure provisioning

---

2. High-Level Backend Architecture



---

3. Backend Objectives

Primary Objectives

3.1 Real-Time Infrastructure

Provide low-latency synchronization for:

- Messaging
- Presence
- Channel updates
- Workspace management
- Notifications

3.2 Persistent Voice Communication

Provide Discord-like voice rooms with:

- Instant join
- Persistent channels
- Low-latency routing
- Multi-user concurrency
- High voice clarity

3.3 Cost-Efficient Scaling

The backend must:

- Avoid persistent VM usage
- Use serverless architecture wherever possible
- Scale only during active traffic
- Reduce idle infrastructure costs

3.4 Enterprise-Grade Security

Implement:

- JWT validation
- Role-based authorization
- Workspace isolation
- Encrypted file access
- Webhook verification
- Signed upload URLs

---

4. Core Technology Stack

Layer	Technology
Authentication	Firebase Auth
Database	Cloud Firestore
Realtime Events	Firestore Listeners
Media Routing	Amazon Chime SDK
Storage	AWS S3
API Layer	AWS API Gateway
Compute	AWS Lambda
Billing	Razorpay
Queue/Event Processing	AWS EventBridge / SQS
Notifications	Firebase Cloud Messaging
Monitoring	AWS CloudWatch + Firebase Analytics
CDN	CloudFront
Secrets Management	AWS Secrets Manager

---

5. Authentication & Identity System

5.1 Authentication Provider

Firebase Authentication Methods

Supported login methods:

- Email + Password
- Google OAuth
- GitHub OAuth

- Magic Link Authentication (Future)

5.2 JWT Session Flow

User Login
↓
Firebase Auth Generates JWT
↓
Frontend Stores Token Securely
↓
Token Sent To Backend APIs
↓
Lambda Verifies JWT
↓
Access Granted

5.3 Security Requirements

Mandatory

- JWT expiration validation
- Refresh token rotation
- Device session tracking
- Suspicious login detection
- Workspace-level access validation

---

6. Workspace Architecture

6.1 Workspace Model

Each workspace acts as a fully isolated tenant.

Isolation Rules

Workspace data must never leak across:

- Channels
- Members
- Files
- Voice sessions
- Billing
- Roles

6.2 Workspace Lifecycle

Creation Flow

User Signup
↓
Workspace Created
↓
Default Channels Created
↓
Owner Role Assigned
↓
Subscription Initialized
↓
Workspace Ready

---

7. Role-Based Access Control (RBAC)

7.1 Permission Architecture

Permissions are stored as:

```
{
  "permissions": {
```

```
    "manage_workspace": true,
    "manage_roles": true,
    "create_channels": true,
    "join_voice": true,
    "upload_files": true,
    "delete_messages": false
  }
}
```

7.2 Permission Categories

Category	Permissions
Workspace	Manage workspace, billing
Channels	Create/delete/edit
Messaging	Send/delete/pin
Voice	Join/mute/kick
Files	Upload/download/delete
Moderation	Ban/remove members

7.3 Role Hierarchy

```
Owner
 ├── Admin
 │    ├── Manager
 │    │    ├── Developer
 │    │    ├── HR
 │    │    └── Designer
 │    └── Moderator
 └── Guest
```

---

8. Firestore Database Design

8.1 Collections Overview

workspaces/
users/
channels/
messages/
voiceSessions/
subscriptions/
roles/
notifications/
activityLogs/
files/

8.2 Workspace Schema


```
{
  "workspaceId": "ws_001",
  "name": "Obsidian Labs",
  "ownerId": "usr_001",
  "tier": "premium",
  "subscriptionStatus": "active",
  "memberCount": 12,
  "storageUsed": 145000000,
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

8.3 User Schema


```
{
  "uid": "usr_001",
  "workspaceId": "ws_001",
  "email": "admin@company.com",
  "displayName": "Aarav",
```

```
  "avatarUrl": "",
  "roleId": "role_admin",
  "status": "online",
  "lastSeen": "timestamp"
}
```

8.4 Channel Schema

```
{
  "channelId": "chn_001",
  "workspaceId": "ws_001",
  "name": "engineering",
  "type": "voice",
  "visibility": "private",
  "allowedRoles": [
    "admin",
    "developer"
  ],
  "chimeMeetingId": "meeting_123"
}
```

8.5 Message Schema

```
{
  "messageId": "msg_001",
  "channelId": "chn_001",
  "senderId": "usr_001",
  "content": "Deployment completed",
  "attachments": [],
  "createdAt": "timestamp",
  "edited": false
}
```

---

9. Voice Infrastructure (Amazon Chime SDK)

9.1 Voice System Requirements

Features

- Persistent voice rooms
- Noise suppression
- Screen sharing
- Multi-user audio
- Voice presence
- Automatic reconnection

9.2 Voice Join Flow

User Clicks Voice Channel
↓
Frontend Sends Join Request
↓
Lambda Validates User
↓
Fetch/Create Chime Meeting
↓
Generate Attendee Token
↓
Return Credentials
↓
Client Connects To Chime

9.3 Voice Session Lifecycle

Idle Cleanup

Voice sessions automatically terminate if:

- No active users remain
- Timeout exceeds 10 minutes
- Workspace becomes inactive

---

10. File Storage System (AWS S3)

10.1 Storage Design

Each workspace receives isolated S3 prefixes:

/workspaces/ws_001/
/workspaces/ws_002/

10.2 Upload Pipeline

Client Requests Upload URL
↓
Backend Verifies Limits
↓
Signed S3 URL Generated
↓
Client Uploads Directly To S3
↓
Metadata Stored In Firestore

10.3 File Restrictions

Tier	Max File Size

Gold	25 MB
Premium	100 MB
Deluxe	500 MB

---

11. Subscription & Billing System

11.1 Subscription Tiers

Feature	Gold	Premium	Deluxe
Members	15	50	150
Storage	5 GB	25 GB	100 GB
Voice Quality	Standard	High-Fi	Spatial
Guest Access	No	No	Yes
Custom Roles	Limited	Full	Advanced

11.2 Razorpay Integration

Billing Flow

User Selects Plan
↓
Razorpay Order Created
↓
Payment Completed
↓
Webhook Triggered
↓
Signature Verified
↓

Workspace Tier Updated

11.3 Webhook Security

Mandatory checks:

- HMAC SHA256 verification
- Replay attack prevention
- Timestamp validation
- Event deduplication

---

12. API Gateway Design

12.1 API Structure

/api/v1/auth/
/api/v1/workspaces/
/api/v1/channels/
/api/v1/messages/
/api/v1/voice/
/api/v1/files/
/api/v1/payments/
/api/v1/admin/

12.2 Example APIs

Join Voice Channel

POST `/api/v1/voice/join`

Request

```
{
  "workspaceId": "ws_001",
  "channelId": "chn_001",
  "userId": "usr_001"
}
```

Response

```
{
  "meetingId": "meeting_123",
  "attendeeId": "att_001",
  "joinToken": "jwt_token",
  "mediaRegion": "ap-south-1"
}
```

---

13. Real-Time Presence System

13.1 Presence Types

Status	Meaning
Online	Active
Away	Idle
Busy	DND
Offline	Disconnected

13.2 Presence Tracking

Presence updates every:

- 30 seconds heartbeat
- User activity changes
- Voice joins/leaves

---

14. Notification Infrastructure

14.1 Notification Types

- Mentions
- Workspace invites
- Voice call alerts
- Billing alerts
- Role updates
- File uploads

14.2 Delivery Channels

Type	Channel
Push	Firebase FCM
Email	AWS SES
In-App	Firestore
SMS (Future)	Twilio

---

15. Backend Security Architecture

15.1 Security Layers

Layer 1 — Authentication

Firebase JWT validation

Layer 2 — Authorization

RBAC permission checks

Layer 3 — Data Isolation

Workspace filtering

Layer 4 — Transport

HTTPS + TLS 1.3

Layer 5 — Storage

Signed URLs + encrypted S3 objects

15.2 Rate Limiting

Endpoint	Limit
Login	10/min
Messages	60/min
Voice Join	20/min
Uploads	15/min

---

16. Monitoring & Observability

16.1 Monitoring Stack

Tool	Purpose
CloudWatch	Lambda metrics

Firebase Analytics	User activity
Sentry	Error tracking
AWS X-Ray	API tracing

16.2 Critical Alerts

- Payment failures
- Lambda crashes
- Database quota spikes
- Voice service outages
- Unauthorized access attempts

---

17. Scalability Strategy

17.1 Horizontal Scalability

The system scales automatically via:

- Lambda concurrency
- Firestore auto-scaling
- Chime elastic media routing
- S3 elastic storage

17.2 Multi-Region Expansion

Future deployment regions:

Region	Purpose
Mumbai	India Primary

Singapore	Asia Backup
Frankfurt	Europe

---

18. Disaster Recovery Strategy

18.1 Backup Policy

Resource	Backup Frequency
Firestore	Daily
S3	Continuous
Billing Data	Realtime
Logs	Hourly

18.2 Recovery Objectives

Metric	Target
RPO	< 15 Minutes
RTO	< 1 Hour

---

19. DevOps & CI/CD

19.1 Deployment Pipeline

GitHub Push
↓

GitHub Actions
↓
Run Tests
↓
Build Lambda Packages
↓
Deploy To AWS
↓
Deploy Firebase Rules

---

20. Backend Cost Optimization

20.1 Cost Reduction Strategies

Firebase

- Minimize document reads
- Use batched writes
- Cache presence states

AWS

- Lambda over EC2
- Chime only during active sessions
- S3 lifecycle policies
- CloudFront caching

---

21. Future Backend Roadmap

Phase 2

- AI Meeting Summaries
- Voice Recording

- Workspace Analytics

Phase 3

- Video Conferencing
- AI Moderation
- Shared Whiteboards

Phase 4

- Multi-workspace federation
- API marketplace
- Plugin ecosystem

---

22. Technical Risks

Risk	Mitigation
Firestore read explosion	Aggressive caching
Voice latency	Regional routing
Billing fraud	Webhook validation
Abuse/spam	Rate limiting
Large storage costs	Lifecycle cleanup

---

23. Production Readiness Checklist

Infrastructure

- Multi-env deployment
- Secret rotation
- Rate limiting
- Monitoring dashboards

Security

- JWT validation
- RBAC testing
- Signed uploads
- Webhook verification

Performance

- Load testing
- Voice concurrency testing
- Firestore optimization

---

24. Final Backend Summary

Obsidian’s backend is designed as:

- Fully serverless
- Highly scalable
- Real-time optimized
- Cost-efficient for Indian startups
- Secure multi-tenant infrastructure
- Enterprise-ready architecture

The Firebase + AWS hybrid model enables:

- Extremely low idle cost
- Massive concurrency support
- Production-grade voice infrastructure
- Simplified deployment operations
- Future scalability into enterprise collaboration ecosystems